

Bitwise Optimizations in C++: A Low Latency Developer's Guide

Boost C++ performance with bitwise tricks for latency-sensitive code.

12 min read · 20 hours ago



Sagar

Following ▾



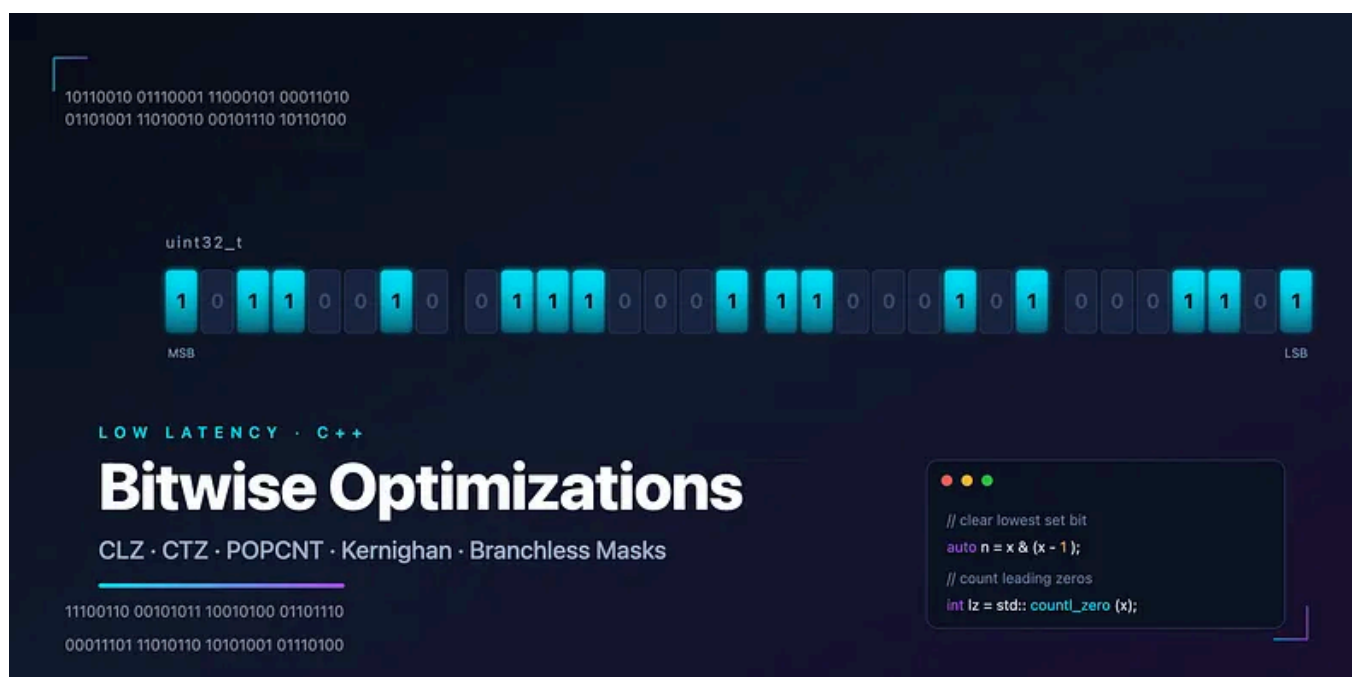
Listen



Share



More



Bitwise operations aren't magic performance dust. In a hot path, they're often simply the shortest distance between your intent and the silicon — a way to express “find the highest bit,” “is this a power of two,” or “count these flags in parallel” without asking the compiler to do guesswork on your behalf.

This post is a practical, modern-C++ tour of the bitwise toolkit I actually reach for when I'm shaving cycles out of a matching engine, a packet parser, or an inference kernel. We'll cover the classic intrinsics, the C++20 `<bit>` header that finally made

this stuff portable, and — importantly — **when not to bother**, because your compiler already did it for you.

Bitwise Optimizations: The Honest Pitch

Let's deflate a myth first. You'll often read advice like:

"Use `x << 3` instead of `x * 8` — it's faster."

That advice is 25 years out of date. Any optimizing compiler strength-reduces `x * 8` to a shift (or a `leaq` on x86) automatically. Writing `x << 3` in your source for speed reasons does nothing except annoy the next person reading your code. Write what you mean.

So where does the real value of bitwise programming come from?

- **Expressing binary structure directly.** `x & (x - 1) == 0` is the power-of-two test. No amount of multiplication makes that cleaner.
- **Mapping to specialized CPU instructions** — `LZCNT`, `TZCNT`, `POPCNT`, `PDEP`, `PEXT`, `BLSR` — that the compiler won't invent for you from ordinary code.
- **Eliminating branches in hot loops** where a mispredict costs you 15+ cycles. Bit masks are the foundation of branchless code.
- **Packing state** into a single register so you can query dozens of booleans with one `AND`.

Keep that framing in mind: bitwise tricks are about *intent and instruction selection*, not about outsmarting the multiplier.

Ground rules:

1. Use unsigned types. Signed shifts and overflow are a UB swamp.
2. Suffix literals with `u` / `ull`: `1u << 31`, `1ull << 63`.
3. `&` is bitwise, `&&` is logical. Enable `-Wall`. Read the warnings.
4. Prefer `<bit>` (C++20) where available, intrinsics otherwise.

The Modern Way: `<bit>` in C++20

If you're on C++20 or newer, most of this article collapses into a handful of standard-library calls. The `<bit>` header is portable, `constexpr`-friendly, well-defined at zero in most cases, and backed by the same intrinsics you'd have reached for anyway.

Check this medium post for deep-dive: [The `<bit>` Header in C++20: Clean, Portable Bit Manipulation](#)

```
#include <bit>
#include <cstdint>

uint32_t x = 0b0010'1100;

int pc  = std::popcount(x);      // 3
int lz  = std::countl_zero(x);   // leading zeros (defined for x == 0)
int tz  = std::countr_zero(x);   // trailing zeros (defined for x == 0)
int lo  = std::countl_one(x);    // leading ones
int w   = std::bit_width(x);     // position of MSB + 1 → floor(log2(x)) + 1
bool p2 = std::has_single_bit(x); // true iff x is a power of two
uint32_t ceil_p2 = std::bit_ceil(x); // next power of two >= x
uint32_t floor_p2 = std::bit_floor(x); // largest power of two <= x
```

These should be your **first choice** in modern code. They work on every **unsigned** integer type, they're `constexpr`, and the compiler emits the same instructions the intrinsics would.

I'll show the intrinsic-level equivalents in the sections below for three reasons:

- you may be stuck on C++17,
- you may be interfacing with SIMD mask registers that `<bit>` doesn't cover, and
- understanding what's underneath makes you a better debugger when something doesn't vectorize.

Intrinsic Bitwise Functions

Intrinsics are compiler built-ins that (usually) translate to a single CPU instruction. Before C++20, they were the only portable-ish way to reach these instructions.

OPERATION	GCC / CLANG	MSVC
Count leading zeros	<code>__builtin_clz</code>	<code>__lzcnt</code> / <code>_BitScanReverse</code>
Count trailing zeros	<code>__builtin_ctz</code>	<code>_tzcnt_u32</code> / <code>_BitScanForward</code>
Popcount	<code>__builtin_popcount</code>	<code>__popcnt</code>
Parity	<code>__builtin_parity</code>	—

Instruction availability is not free:

A critical caveat that trips up juniors (and a depressing number of seniors):

- **LZCNT**, **TZCNT**, and **POPCNT** are not **universal**. They're gated behind CPU feature flags (**ABM**, **BMI1**, **SSE4.2**).
- On GCC/Clang you typically need `-mlzcnt -mbmi -mpopcnt`, or better: `-march=native` / `-march=x86-64-v3`.
- On MSVC, `__lzcnt` on a CPU without ABM silently decodes as **BSR** on some paths and as `rep bsr` on others. Behavior and performance differ.
- If you ship binaries that run on old hardware, you need either a compile-time target floor, runtime CPU dispatch via `cpuid`, or a software fallback.

`std::countl_zero` handles this for you portably — another reason to prefer `<bit>`.

A defensive wrapper pattern for pre-C++20 code:

```
#include <cstdint>

#if defined(_MSC_VER)
#include <intrin.h>
inline int aussie_clz32(uint32_t x) {
    unsigned long i;
```

```

    // _BitScanReverse is available everywhere; LZCNT needs ABM.
    return _BitScanReverse(&i, x) ? 31 - (int)i : 32;
}
inline int aussie_ctz32(uint32_t x) {
    unsigned long i;
    return _BitScanForward(&i, x) ? (int)i : 32;
}
inline int aussie_popcount32(uint32_t x) { return (int)__popcnt(x); }
#else
inline int aussie_clz32(uint32_t x) { return x ? __builtin_clz(x) : 32; }
inline int aussie_ctz32(uint32_t x) { return x ? __builtin_ctz(x) : 32; }
inline int aussie_popcount32(uint32_t x) { return __builtin_popcount(x); }
#endif

```

Undefined behavior alert: `__builtin_clz(0)` and `__builtin_ctz(0)` are UB. Guard the zero case or use `<bit>`.

CLZ — Count Leading Zeros

What it does: Counts zero bits to the *left* of the highest set bit. It's the fastest way to find the most significant bit (MSB), which is the foundation of integer log2, bit-width, and power-of-two rounding.

```

x = 0000 0000 0000 0000 0000 0000 0001 0110    (decimal 22)
      ^-- 27 leading zeros --^
CLZ(22) = 27

```

Modern C++:

```

#include <bit>
int msb_pos = std::bit_width(x) - 1;    // defined for x == 0 (returns -1)

```

Pre-C++20:

```
// Precondition: x != 0
inline int msb_position(uint32_t x) {
    return 31 - aussie_clz32(x);
}
```

CPU instructions: **LZCNT** (defined on zero) and the older **BSR** (undefined on zero).

CTZ — Count Trailing Zeros

What it does: Counts zero bits to the *right* of the lowest set bit. Use this for iterating over set bits — bitboard chess engines, scheduler ready-queues, event dispatch tables.

```
x = 0000 0000 0000 0000 0000 0000 0001 0000    (decimal 16)
                                ^----^
CTZ(16) = 4
```

Idiom: iterate every set bit:

```
#include <bit>

template <typename F>
void for_each_set_bit(uint64_t mask, F&& fn) {
    while (mask) {
        int i = std::countr_zero(mask);
        fn(i);
        mask &= mask - 1;    // clear lowest set bit (see Kernighan, §7)
    }
}
```

That inner pattern — **countr_zero** followed by **x &= x - 1** — compiles to roughly two instructions per bit on modern x86 (**TZCNT** + **BLSR**). It's the tightest "iterate set bits" loop you can write portably.

CPU instructions: **TZCNT** (safe on zero) and **BSF** (UB on zero).

Popcount — Counting Set Bits

What it does: Counts how many **1** bits are set. Useful for cardinality of bit-sets, Hamming distance, and as part of XNOR-popcount inner products in quantized neural nets.

The didactic version (for reference only):

```
int popcount_naive(uint32_t x) {  
    int n = 0;  
    while (x) { n += (x & 1u); x >>= 1; }  
    return n;  
}
```

Kernighan version: Each iteration of $x \&= x - 1$ clears one set bit, so we loop only as many times as there are ones:

```
int popcount_kernighan(uint32_t x) {  
    int n = 0;  
    while (x) { x &= x - 1; ++n; }  
    return n;  
}
```

In modern C++20 you should use this:

```
#include <bit>  
int n = std::popcount(x);
```

[Open in app](#) ↗

loop `std::popcount` one word at a time — you'll serialize on the latency. Instead:

- Unroll 4–8x so multiple `POPCNT` instructions are in flight.
- On AVX-512 machines, `VPOPCNTQ` pops eight 64-bit lanes at once.
- For large buffers without AVX-512, the Harley-Seal algorithm (a carry-save adder tree) outperforms per-word `POPCNT`.

This is the kind of detail that actually matters in a hot loop. “Use the intrinsic” is step one; “feed the intrinsic enough independent work to saturate the pipeline” is step two.

Kernighan's Bit Trick

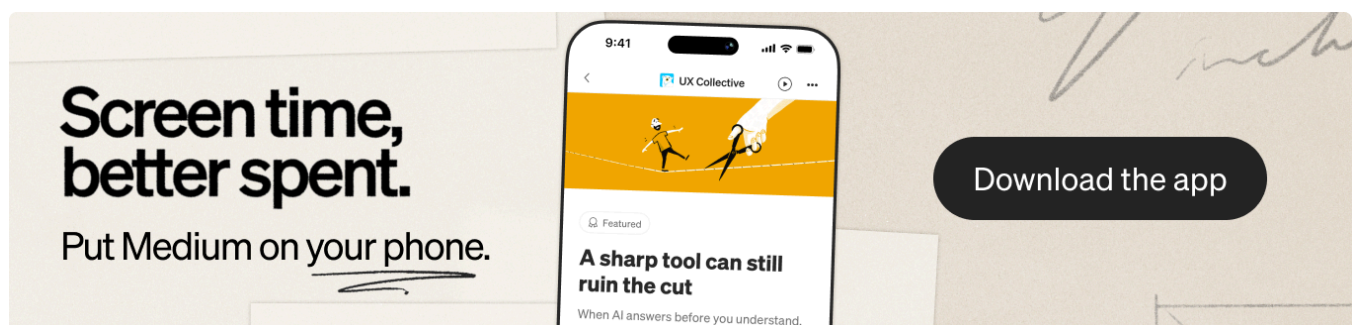
Brian Kernighan's identity, in one line:

```
x & (x - 1) clears the lowest set bit of x.
```

Tiny observation, enormous payoff. It powers a whole family of idioms:

```
x & (x - 1)           // clear lowest set bit
x & -x                // isolate lowest set bit (BLSI on BMI1)
x ^ (x - 1)           // mask of lowest set bit and all bits below (BLSMSK)
x | (x + 1)           // set lowest zero bit
(x & (x - 1)) == 0     // true if x is 0 or a power of two
```

That last one is probably the most-used bit trick in systems programming.



Finding the highest bit (the old-school way):

Before intrinsics were portable, this “bit smear” was the canonical way to isolate the MSB:

```
// Returns a value with ONLY the highest set bit of x kept.  
// Example: 0b0010'1100 → 0b0010'0000  
inline uint32_t highest_bit_smear(uint32_t x) {  
    x |= x >> 1;  
    x |= x >> 2;  
    x |= x >> 4;  
    x |= x >> 8;  
    x |= x >> 16;  
    return x - (x >> 1);  
}
```

On modern hardware `std::bit_floor(x)` is faster and clearer. Keep the smear in your back pocket for environments where intrinsics aren't available (embedded, custom DSPs, shader code).

Fast NOR / NAND / XNOR

C++ gives you `&`, `|`, `^`, `~` — and nothing else. NAND, NOR, and XNOR are always two operations:

```
inline uint32_t bw_nand(uint32_t x, uint32_t y) { return ~(x & y); }  
inline uint32_t bw_nor (uint32_t x, uint32_t y) { return ~(x | y); }  
inline uint32_t bw_xnor(uint32_t x, uint32_t y) { return ~(x ^ y); }
```

In practice, this almost never matters. Two dependent ALU ops retire in a cycle or two, and the compiler can usually fuse the `~` with surrounding logic or hoist it out of loops.

When the hardware does have a native op

On AVX-512 mask registers (the `k` registers), bitwise logic *is* a single instruction:

- `KAND`, `KOR`, `KXOR`, `KXNOR`, `KANDN` — one-cycle ops on mask registers.

- **VPTERNLOGD/Q** — compute any three-input boolean function on vector registers in one instruction. Pick a truth-table byte, and the hardware evaluates it for you.

If you're on AVX2 or plain BMI1/BMI2, the more practical “missing operator” hardware is:

- **ANDN** (`__andn_u32` , or just write `~x & y` — the compiler emits **ANDN** automatically).
- **BLSR** (clear lowest bit), **BLSI** (isolate lowest bit), **BLSMSK** (mask up to lowest bit).
- **PDEP** / **PEXT** — parallel bit deposit/extract. Worth their own post.

Focus your attention on BMI1/BMI2 before AVX-512 mask tricks. You'll get more mileage out of them in general low-latency code.

Fast LOG2 of Integers

Integer log2 is just “position of the highest set bit.”

The slow way:

```
#include <cmath>
int log2_slow(uint32_t u) { return (int)std::log2f((float)u); }
```

This goes through the FPU, can mis-round at boundaries (e.g. `log2f(8.0f)` computed as `2.9999...`), and is easily 10× slower than the integer approach.

The fast way (C++20):

```
#include <bit>
// Precondition: u != 0
inline int ilog2(uint32_t u) { return std::bit_width(u) - 1; }
```

Pre-C++20:

```
inline int ilog2(uint32_t u) { return 31 - aussie_clz32(u); }
```

Sanity table:

<code>`u`</code>	BINARY	<code>`BIT_WIDTH`</code>	<code>`ILOG2`</code>
1	<code>`...0001`</code>	1	0
4	<code>`...0100`</code>	3	2
7	<code>`...0111`</code>	3	2
8	<code>`...1000`</code>	4	3

Note: don't use CTZ

`CTZ`, `BSF`, and `ffs` give you the lowest set bit, not the highest. For `x = 12 (0b1100)`, CTZ-based "log2" returns 2 but the real floor-log2 is 3. Mixing these up is a classic bug.

Fast Largest Power-of-Two

“Round down to the largest power of two that fits.” Also stated as “zero every bit except the MSB.”

```
22 (0b10110) → 16 (0b10000)
31 (0b11111) → 16
32 (0b100000) → 32
```

C++20:

```
#include <bit>
uint32_t p = std::bit_floor(x);    // largest pow2 <= x (returns 0 for x == 0)
uint32_t c = std::bit_ceil(x);     // smallest pow2 >= x
bool is_p2 = std::has_single_bit(x);
```

Pre-C++20:

```
// Precondition: u != 0
inline uint32_t highest_pow2(uint32_t u) {
    return 1u << (31 - aussie_clz32(u));
}

// Smallest power-of-two >= u, without intrinsics
inline uint32_t next_pow2(uint32_t x) {
    if (x <= 1) return 1;
    --x;
    x |= x >> 1;
    x |= x >> 2;
    x |= x >> 4;
    x |= x >> 8;
    x |= x >> 16;
    return x + 1;
}
```

These show up constantly in hash-table sizing, ring-buffer allocation, and GPU work-group rounding. Cache the pow2 capacity and replace `x % capacity` with `x & (capacity - 1)` — that one substitution is worth the whole article in a tight hash probe loop.

Branchless Programming with Bit Masks

Here's where bit tricks stop being cute and start being the reason your code fits in a microsecond budget.

Modern CPUs mispredict branches. On a mispredict, you eat 15–20 cycles of pipeline flush. In a hot path with unpredictable data (e.g. sorting random inputs, parsing

mixed packet types), replacing a branch with an arithmetic/bitwise expression can be a 2–5× improvement even though you’re doing “more work.”

The core idea: sign-extension gives you a mask

For any signed integer comparison, $-(a < b)$ is $0xFFFF\dots F$ if true, 0 if false. That's a ready-made bitmask.

```
// Branchless min / max (works for signed ints with no overflow on a-b)
inline int branchless_min(int a, int b) {
    return b ^ ((a ^ b) & -(a < b));
}
inline int branchless_max(int a, int b) {
    return a ^ ((a ^ b) & -(a < b));
}
```

Worth noting: modern compilers will lower `std::min / std::max` to `CMOV` (a conditional move) on their own, which is also branchless. **Measure before hand-rolling.** The XOR trick matters more in scalar code that the compiler *can't* prove is safe to transform, or in **SIMD** lanes where `CMOV` doesn't exist.

Conditional add / subtract without branches

```
// if (cond) x += delta;
int32_t mask = -(int32_t)cond;      // 0 or 0xFFFFFFFF
x += (delta & mask);
```

This pattern eliminates the branch and keeps the data path predictable. I use it in order-book updates where the condition is effectively random (incoming message types).

Absolute value without a branch

```
inline int32_t branchless_abs(int32_t x) {
    int32_t mask = x >> 31;      // arithmetic shift: 0 or -1
    return (x ^ mask) - mask;
}
```

(For portable “arithmetic shift on signed” behavior, GCC/Clang/MSVC all do what you want on two’s-complement platforms — which is now mandated by C++20.)

Clamp to zero on negative — “ReLU”

```
inline int32_t relu(int32_t x) {  
    return x & ~(x >> 31);           // zero if sign bit set  
}
```

Used everywhere in quantized neural-net inference. One instruction after optimization.

Conditional select without `CMOV`

```
// Return a if cond, else b — no branch, no CMOV, pure bit ops.  
inline uint32_t select(bool cond, uint32_t a, uint32_t b) {  
    uint32_t m = ~(uint32_t)cond;    // all 1s or all 0s  
    return (a & m) | (b & ~m);  
}
```

This version is what you want inside SIMD code, where scalar `CMOV` isn't available but the same pattern extends naturally to vector masks.

The crucial context here is:

Branchless isn’t always faster. If the branch is *highly predictable* (>99%), the predictor makes it essentially free, and the branchless version — which always does both halves of the work — is slower. The wins come when:

- The condition is genuinely unpredictable, or
- You’re inside a SIMD lane where branches don’t exist, or
- You need constant-time behavior (crypto, timing-attack-resistant code).

Profile. Always profile. `perf stat -e branch-misses` is your friend.

Cheat Sheet

Tape this to your monitor:

```
#include <bit>

std::popcount(x)           // count set bits
std::countl_zero(x)        // count leading zeros
std::countr_zero(x)        // count trailing zeros
std::bit_width(x)          // MSB position + 1 (== floor(log2) + 1)
std::bit_floor(x)          // largest pow2 <= x
std::bit_ceil(x)           // smallest pow2 >= x
std::has_single_bit(x)     // is power of two

x & (x - 1)                // clear lowest set bit
x & -x                     // isolate lowest set bit
(x & (x - 1)) == 0         // power-of-two check (true for 0 too)
x & (N - 1)                // fast modulo when N is pow2
-(int)cond                 // bool → all-ones / all-zeros mask
(a & m) | (b & ~m)         // branchless select via mask m
```

Final Thoughts

Three things I want you to walk away with:

1. Prefer `<bit>`. It's portable, `constexpr`, well-defined at zero, and it compiles to the same instructions your intrinsics would. Reach for `__builtin_*` only when you have a reason.
2. The compiler is smarter than 2005. Don't write `x << 3` for speed. Don't hand-roll `CMOV`. Write the clearest expression of your intent first, read the disassembly, and only intervene where the compiler demonstrably missed something.
3. The real value of bitwise programming is expressiveness and branch elimination, not squeezing cycles out of shifts. Bit masks let you compute what would otherwise be conditional logic, which is the single biggest lever you have against branch mispredicts in data-dependent hot paths.